Project – Airline Reservation System

Object Oriented Programming

(CSEG1044)

Phase 5: Testing

Members:-

Yash Jangir – 590014858

Parth Choudhary-590016913

Dushyant Singh-590012592

Soham Sharma-590017041

Anuj Pandey-59001725

1. Executive Summary and Phase Goal

Goal of the Phase:

The primary goal of Phase 5 is to systematically verify that the Airline Reservation System functions correctly across all layers (UI, Service, DAO, and Database). This phase ensures that the application meets the requirements defined in Phase 1, handles data securely as designed in Phase 3, and provides a stable, user-friendly experience as built in Phase 4.

2. Test Strategy and Scope

To ensure comprehensive coverage, the testing strategy is divided into three distinct levels:

- **Unit Testing:** Validating individual DAO methods (e.g., verifying `RouteDAO.addRoute()` correctly inserts data) and Service logic (e.g., checking flight capacity constraints).
- **Integration Testing:** Ensuring the Java Swing UI correctly passes data to the Service layer, which in turn successfully triggers the JDBC connections in the DAO layer without data loss.
- **System (End-to-End) Testing:** Simulating real-world user journeys (Admin and Passenger) from login to final seat allocation.

3. Core System Test Cases

The following test cases map directly to the acceptance criteria established in Phase 1 and the UI components built in Phase 4.

Admin Workflows

Test ID	Module	Test Scenario	Expected Result	Status
TC-01	Route Mgt	Admin adds a new valid Route (Origin: JFK, Dest: LHR).	System saves route to DB and shows "Route added successfully" `JOptionPane`.	**PASS**
TC-02	Route Mgt	Admin attempts to add a Route with empty fields.	System blocks submission and displays a localized validation error message.	**PASS**
TC-03	Flight Mgt	Admin schedules a flight with a valid Route ID and Future Date.	Flight is created in `flights` table; visible on Passenger dashboard.	**PASS**

Test ID	Module	Test Scenario	Expected Result	Status
TC-04	Flight Mgt	Admin schedules a flight using a non-existent Route ID.	Service layer catches the invalid foreign key; UI alerts Admin of invalid Route.	**PASS**

Passenger Workflows

Test ID	Module	Test Scenario	Expected Result	Status
TC-05	Booking	Passenger selects an available flight and clicks "Book".	Booking record created in DB with "PENDING" status; success popup shown.	**PASS**
TC-06	Booking	Passenger attempts to book the *same* flight twice.	BookingService detects duplicate; UI displays "You have already booked this flight."	**PASS**
TC-07	Booking	Passenger attempts to book a flight that has reached maximum capacity.	System rejects booking; UI displays "Flight is fully booked."	**PASS**

Confirmation & Scheduling

Test ID	Module	Test Scenario	Expected Result	Status
TC-08	Confirmation	Admin changes a pending booking status to "CONFIRMED".	Database updates status to CONFIRMED; UI reflects changes immediately.	**PASS**
TC-09	Seat Alloc.	Admin assigns an available seat (e.g., 12A) to a confirmed booking.	Seat is successfully linked to the Booking ID in the DB.	**PASS**
TC-10	Seat Alloc.	Admin tries to assign a seat that is already taken by another passenger.	System rejects assignment; UI displays "Seat conflict: Please choose another seat."	**PASS**

---

4. Defect Logging and Resolution

During the testing phase, a few critical issues were identified and resolved to maintain architectural integrity:

- **Bug 01: UI Freezing on Database Connection**
  - *Issue:* The Swing GUI would briefly freeze when clicking "Book Flight" if the database connection was slow.
  - *Resolution:* Moved the database initialization logic to a background thread (using `SwingWorker`) so the main Event Dispatch Thread (EDT) remains responsive.
- **Bug 02: Data Type Mismatch Exceptions**
  - *Issue:* Entering letters into the "Passenger ID" or "Route ID" text fields caused SQL/Java exceptions to crash the application.
  - *Resolution:* Implemented strict input validation at the UI layer (`try-catch` blocks for `Integer.parseInt()`) to warn the user before the data reaches the Service/DAO layers.
- **Bug 03: Orphaned Records**
  - *Issue:* Deleting a Route that had active Flights associated with it caused a database constraint error.
  - *Resolution:* Enforced `ON DELETE RESTRICT` in the MySQL schema (Phase 3) and added a user-friendly error message in the Service layer explaining that active flights must be cancelled before a route can be deleted.